

ĐẠI HỌC QUỐC GIA TP HCM
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



ĐỒ ÁN LẬP TRÌNH SOCKET

Môn học: Mạng máy tính

Sinh viên thực hiện:

24120387: Trần Nhật Nam

24120406: Nguyễn Huỳnh Huy Phát

24120357: Nguyễn Nhân Kiệt

Giáo viên hướng dẫn:

Lê Hà Minh

Nguyễn Thanh Quân

Mục lục

1	Giới thiệu	1
1.1	Danh sách thành viên	1
1.2	Giới thiệu về đồ án	1
1.3	Mục đích	1
2	GitHub Repository	2
3	Cấu trúc hệ thống	2
3.1	Tổng quan	2
3.2	Giao thức sử dụng	2
3.3	Cấu trúc mã nguồn	3
3.3.1	Phía Server	3
3.3.2	Phía Client	3
3.3.3	Tập tin dùng chung	4
3.4	Luồng hoạt động	4
3.4.1	Cách chạy chương trình	4
3.4.2	Luồng hoạt động	4
4	Kết quả	6
4.1	Chạy video mẫu movie.Mjpeg	6
4.2	Chạy video HD với định dạng chuẩn .mjpeg	7
4.3	Kiểm tra Buffering và HD-Streaming	8
5	Kết luận	8
5.1	Kết quả đạt được	8
5.2	Những hạn chế	9
6	Tài liệu tham khảo	9

1 Giới thiệu

1.1 Danh sách thành viên

STT	MSSV	Họ và tên	Email
1.	24120387	Trần Nhật Nam	24120387@student.hcmus.edu.vn
2.	24120357	Nguyễn Nhân Kiệt	24120357@student.hcmus.edu.vn
3.	24120406	Nguyễn Huỳnh Huy Phát	24120406@student.hcmus.edu.vn

Bảng 1: Danh sách thành viên

1.2 Giới thiệu về đề án

Đây là một đề án về **Video Streaming Client-Server** sử dụng ngôn ngữ lập trình **Python** với các thư viện dùng để lập trình socket, xử lý đồ họa. Đề án sử dụng chủ yếu hai giao thức chính.

- **RTSP**: chạy trên nền **TCP**, đảm bảo độ tin cậy khi truyền các lệnh **SETUP**, **PLAY**, **PAUSE**, **TEARDOWN**.
- **RTP**: chạy trên nền **UDP**, tối ưu hóa tốc độ truyền tải video.

Ngoài ra đề án còn có thêm hai chức năng nâng cao khác:

- **Kỹ thuật phân mảnh gói tin (Packet Fragmentation:)** cho phép truyền tải video dung lượng cao (HD).
- **Cơ chế bộ đệm (Buffering)**: Sử dụng hàng đợi để tích lũy dữ liệu, giúp video phát mượt mà.

1.3 Mục đích

1. Củng cố lý thuyết về kiến thức mạng

- Hiểu rõ cơ chế hoạt động của Client-Server.
- Phân biệt được vai trò và cách sử dụng của UDP và TCP.
- Nắm bắt quy chuẩn của gói tin RTP và RTSP

2. Rèn luyện kỹ năng lập trình socket

- Lập trình socket với Python
- Thực hiện được lập trình đa luồng, lắng nghe dữ liệu và cập nhật giao diện.

3. Giải quyết được vấn đề HD-Streaming và Buffering

- Nghiên cứu và cài đặt thuật toán phân mảnh (Packet Fragmentation) để truyền tải video HD qua mạng
- Cài đặt cơ chế bộ đệm giúp video phát mượt mà.

2 GitHub Repository

Link: <https://github.com/nhtnam232/SocketProgrammingProject>

3 Cấu trúc hệ thống

3.1 Tổng quan

Đồ án được xây dựng dựa trên mô hình **Client-Server**, trong đó:

- **Server:** Chịu trách nhiệm xử lý video đầu vào, cắt nhỏ dữ liệu và gửi đi qua mạng.
- **Client:** Chịu trách nhiệm gửi yêu cầu điều khiển, nhận dữ liệu từ Server, lắp ráp và hiển thị.

Hệ thống hoạt động dựa trên cơ chế **Streaming**, cho phép người dùng phát video mà không cần phải tải về máy.

3.2 Giao thức sử dụng

Các giao thức sử dụng:

1. RTSP (Real Time Streaming Protocol)

- Chạy trên nền giao thức TCP.
- Thiết lập và kiểm soát các lệnh từ phía người dùng.

2. RTP (Real-time Transport Protocol)

- Chạy trên nền giao thức UDP.
- Chịu trách nhiệm truyền tải video với tốc độ tối ưu và giảm độ trễ, chấp nhận mất một vài gói tin.

3.3 Cấu trúc mã nguồn

Đồ án được triển khai với ngôn ngữ **Python**, mỗi phần được đảm nhiệm công việc cụ thể.

3.3.1 Phía Server

1. **VideoStream.py**: Có thể đọc hai loại file **.Mjpeg** (custom format của đề bài) và **.mjpeg** (standard format), tách ra từng khung hình của video và xác định số thứ tự khung hình.
2. **Server.py**: Khởi tạo kết nối **TCP**, lắng nghe kết nối từ **Client** và tạo ra các luồng xử lý riêng biệt (**ServerWorker.py**) để phục vụ cho từng **Client**.
3. **ServerWorker.py**: Đây là tập tin xử lý các logic chính của **Server**.
 - Nhận và xử lý các yêu cầu **RTSP** từ phía **Client**.
 - Thực hiện đóng gói và gửi gói tin **RTP** đến Client.
 - Thực hiện chức năng nâng cao **Phân mảnh**, một khung hình sẽ được cắt ra từng gói tin nhỏ có kích thước 1400 bytes để phù hợp với MTU của mạng, gói tin cuối cùng sẽ được đánh dấu **Marker = 1** để Client nhận biết và lắp ráp.

3.3.2 Phía Client

1. **ClientLauncher.py**: Sử dụng thư viện **tkinter** để cài đặt giao diện đồ họa với các nút bấm **SETUP**, **PLAY**, **PAUSE**, **TEARDOWN**. Tiếp nhận các thông số như **IP**, **PORT** của **Server**, cổng để nhận dữ liệu và tên video muốn chạy
2. **Client.py**: là tập tin xử lý các logic chính ở phía **Client**
 - Quản lý giao diện người dùng, gửi yêu cầu **RTSP**, nhận dữ liệu từ **Server**
 - Thực hiện logic của các chức năng nâng cao:

- Luồng **listenRtp** thực hiện việc nhận và gom dữ liệu, kiểm tra bit **Marker** xem có phải là gói tin cuối cùng của khung hình hay chưa.
- Sử dụng cơ chế hàng đợi **queue** để tích lũy các khung hình trước khi phát giúp đảm bảo độ mượt mà, chống hiện tượng giật.

3.3.3 Tập tin dùng chung

1. `RtpPacket.py`: Chủ yếu thực hiện hai chức năng chính là **encode** và **decode**
 - **Encode**: Sử dụng **Bit manipulation** để đóng gói các thông tin vào header 12 bytes của gói tin RTP.
 - **Decode**: Trích xuất thông tin từ header và dữ liệu của gói tin RTP.

3.4 Luồng hoạt động

3.4.1 Cách chạy chương trình

1. **Khởi động Server** Mở Terminal/CMD tại thư mục chứa mã nguồn và video, thực hiện lệnh sau:

```
1 python Server.py <server_port>
```

2. **Khởi động Client** Mở Terminal/CMD khác với Server, thực hiện lệnh sau

```
1 python ClientLauncher.py <server_ip> <server_port> <rtp_port> <  
video_file>
```

3. Khi đó hãy ấn **SETUP** và tiếp tục là **PLAY** tại giao diện đồ họa,
4. Nhấn **PAUSE** để tạm dừng và **TEARDOWN** để kết thúc

3.4.2 Luồng hoạt động

Giai đoạn 1: Thiết lập

- **Client** gửi lệnh **SETUP** cho **Server**, chứa tên file video và cổng UDP mà nó mở để nhận dữ liệu.

- **SERVER** nhận được yêu cầu và kiểm tra sự tồn tại của video, khởi tạo luồng đọc video VideoStream.py và sinh ra một **Session ID**. Sau đó sẽ gửi thông điệp phản hồi, ví dụ RTSP/1.0 200 OK kèm theo **Session ID**, **Client** nhận được sẽ chuyển từ trạng thái **INIT** sang **READY**.

Giai đoạn 2: Truyền tải và xử lý dữ liệu

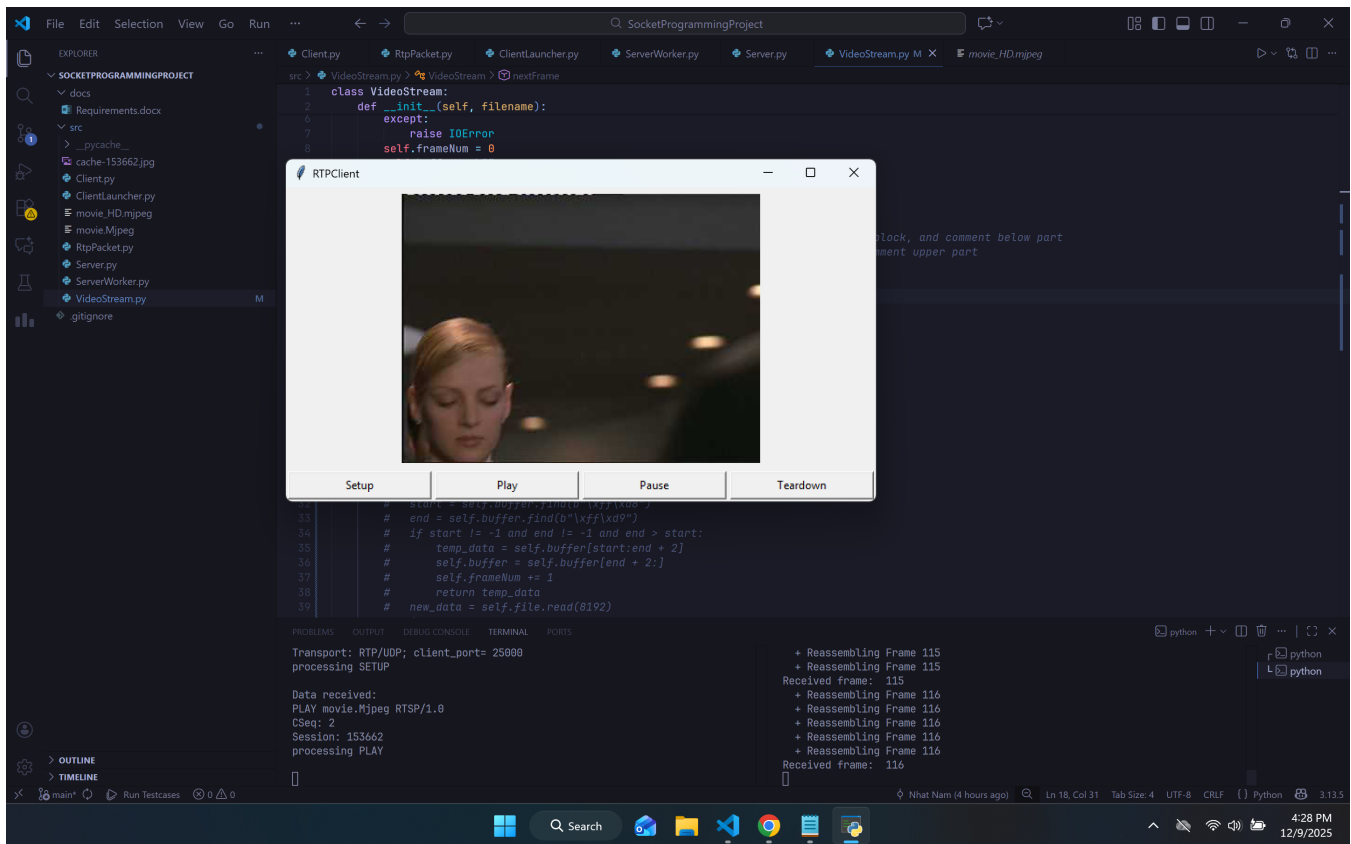
- **Client** gửi lệnh **PLAY**, và kích hoạt luồng nhận tin `listenRtp` và luồng hiển thị `playBuffer`.
- **Server** đọc một khung hình, vì làm theo cơ chế phân mảnh nên **Server** sẽ cắt khung hình thành các gói tin nhỏ và gửi đi, đồng thời, đóng gói gói tin và gán cho từng gói tin bit Marker = 0, nếu là gói cuối cùng của khung hình thì sẽ gán bit Marker = 1.
- **Client** nhận được các gói tin rời rạc, các gói tin nhỏ này được cộng dồn vào biến tạm, nếu gặp gói tin có bit Marker = 1 thì **Client** sẽ biết đây là gói cuối cùng của một khung hình.
- **Cơ chế bộ đệm:** Một khung hình hoàn chỉnh sẽ không được hiển thị ngay mà ở giai đoạn **Pre-Buffering**, nó sẽ được đẩy vào hàng đợi **Queue**, khi giai đoạn kết thúc (tức là đã tích lũy đủ một lượng dữ liệu ví dụ 20 frames) thì một luồng `playBuffer` sẽ lấy từng khung hình trong hàng đợi và hiển thị.

Giai đoạn 3: Kết thúc chương trình

- **Client** gửi lệnh **TEARDOWN** đến **Server**, **Server** đóng luồng đọc file và ngắt kết nối socket, ở phía **Client** thì sẽ xóa file cache ảnh tạm thời.
- Lúc này ở phía **Client** sẽ hiển thị tổng dữ liệu đã nhận và tốc độ trung bình.

4 Kết quả

4.1 Chạy video mẫu movie.Mjpeg



Hình 1

```
8554
>>
Data received:
SETUP movie.Mjpeg RTSP/1.0
CSeq: 1
Transport: RTP/UDP; client_port= 25000
processing SETUP

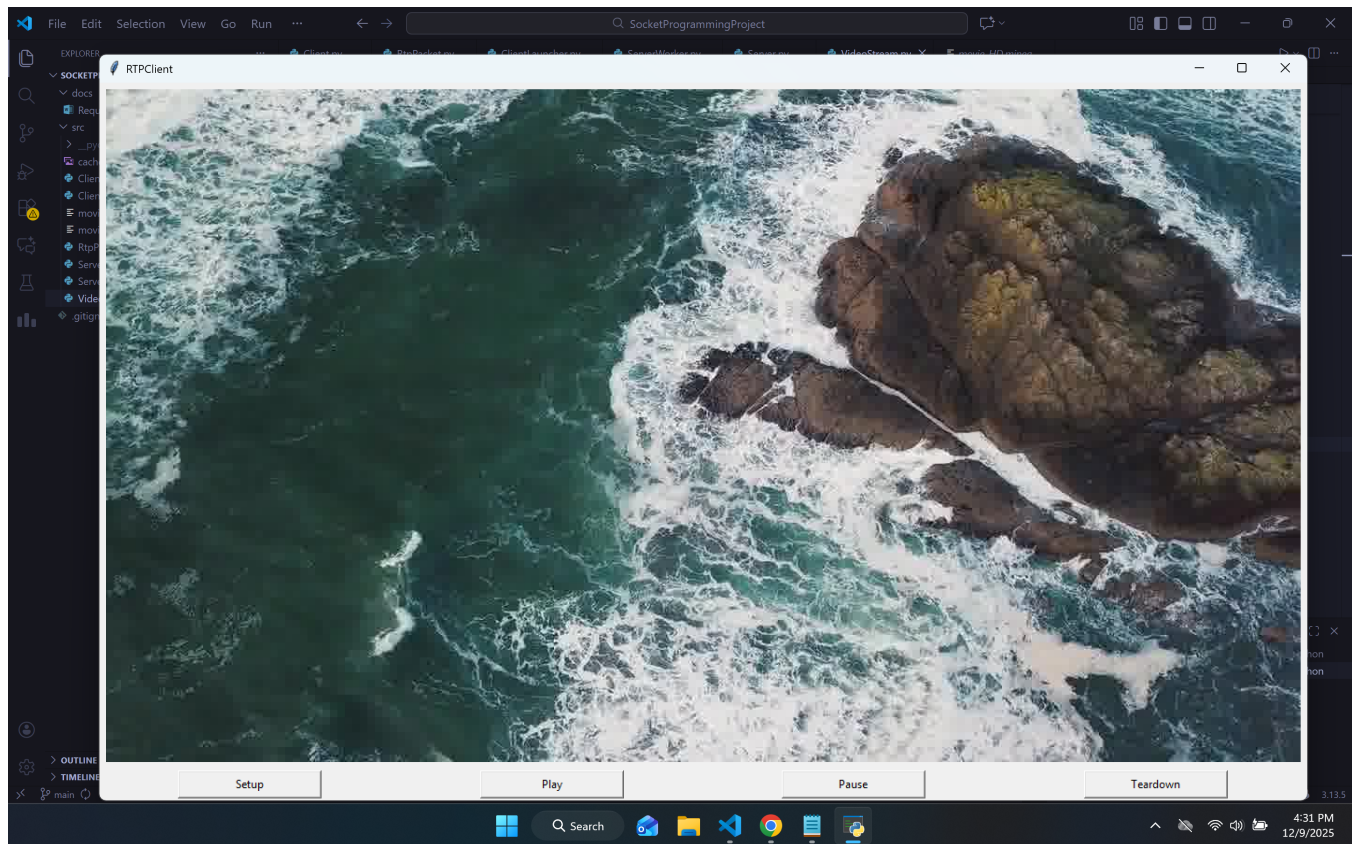
Data received:
PLAY movie.Mjpeg RTSP/1.0
CSeq: 2
Session: 197507
processing PLAY

Data sent:
SETUP movie.Mjpeg RTSP/1.0
CSeq: 1
Transport: RTP/UDP; client_port= 25000
RTP socket opened on port 25000

Data sent:
PLAY movie.Mjpeg RTSP/1.0
CSeq: 2
Session: 197507
+ Reassembling Frame 1
+ Reassembling Frame 1
```

Hình 2

4.2 Chạy video HD với định dạng chuẩn .mjpeg



Hình 3

- Kỹ thuật Phân mảnh và Lắp ráp: Nhóm đã xử lý thành công giới hạn kích thước gói tin của đường truyền mạng, bằng cách chia nhỏ các khung hình video kích thước lớn (HD) tại Server và ghép lại chính xác tại Client dựa trên bit Marker.
- Cơ chế Bộ đệm phía Client: Để khắc phục hiện tượng độ trễ không ổn định của UDP, nhóm đã cài đặt thành công thuật toán bộ đệm. Việc tích lũy dữ liệu trước khi phát và cơ chế hàng đợi giúp video chuyển động mượt mà, loại bỏ hoàn toàn hiện tượng giật hình hay đứng hình khi mạng chậm chạp.

5.2 Những hạn chế

1. Nhóm chúng em chưa thiết kế lại giao diện một cách bắt mắt, tăng trải nghiệm người dùng.

6 Tài liệu tham khảo

1. Computer Networking: A Top-Down Approach. 9th, Jim Kurose and Keith Ross
2. Reading 21: Sockets & Networking
<https://ocw.mit.edu/ans7870/6/6.005/s16/classes/21-sockets-networking/>
3. Github tham khảo:
<https://github.com/jaynavadiya/TF3-Video-Streaming-With-RTSP-And-RTP>